SLCAS

Smart Library Circulation & Automation System

Beginner's Java Guide & Project Study Notes

MIVA Open University · Java Programming Project

Java Syntax | OOP | Architecture | ASCII UML | Code Walkthrough | Viva Q&A

Part 0 — Java Quick-Start: Syntax Essentials

A ground-up brush-through of every Java syntax concept needed to read this project's code. Each topic is tied directly to where it appears in SLCAS.

0.1 Your First Java Program

Every Java application starts with a class containing a main method. Java is case-sensitive; every statement ends with a semicolon.

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
// public  = accessible from anywhere
// static  = belongs to the class, not a specific object
// void    = returns nothing
// String[] args = command-line arguments passed at startup
// In SLCAS: main/LibraryApp.java contains the main() entry point
```

The main() method is the first thing the JVM calls. In SLCAS it triggers: load staff accounts → seed demo data → show login dialog → open main window.

0.2 Primitive Data Types

Primitives store plain values directly in memory — they are not objects and have no methods.

Type	Meaning / Range	Where in SLCAS
int	32-bit whole number	year, totalCopies, availableCopies, staffId
long	64-bit whole number (suffix L)	serialVersionUID = 1L; getDaysOverdue(): long
boolean	true or false only	isAvailable(), isOverdue(), isAdmin()
double	64-bit decimal number	Not directly used (could model late fines)
byte	8-bit integer (-128 to 127)	byte[] hash arrays in SHA-256 hashing
char	Single Unicode character	char[] in JPasswordField (Swing login)

0.3 Variables and Constants

A variable is a named slot whose value can change. A constant uses final — set once, never changed again. static final means it also belongs to the class rather than an object. Constants are written in ALL_UPPER_CASE by convention.

```
// VARIABLE — can be reassigned later
int year = 2024;
String title = "Clean Code";
boolean available = true;
year = 2025;              // OK — just update the value

// CONSTANT (final) — value is locked at assignment forever
final int LOAN_PERIOD_DAYS = 14;              // BorrowRecord.java
private static final int  CACHE_SIZE = 10;   // LibraryDatabase.java
public  static final int  SUPER_ADMIN_ID = 30114413; // AuthManager.java
```

```
LOAN_PERIOD_DAYS = 30;   // COMPILER ERROR: cannot assign to final variable

// 'var' (Java 10+) infers the type automatically:
var list = new ArrayList<LibraryItem>(); // inferred as ArrayList<LibraryItem>
var id   = IDGenerator.generateID();      // inferred as String
```

0.4 Reference Types & String

Everything that is not a primitive is a reference type — the variable holds a memory address, not the value directly. String is the most common. Critical rule: always compare Strings with .equals(), never == (which compares addresses, not content).

```
String name = "Alice";
String padded = "  hello  ".trim();       // "hello"
boolean blank = "".isBlank();              // true

// CORRECT — compares actual content:
if (name.equals("Alice"))   { /* match */ }
// WRONG — compares memory addresses, almost always false:
if (name == "Alice")        { /* unreliable! */ }

// Case-insensitive compare — used throughout LibraryDatabase:
id.equalsIgnoreCase("bk001")  // matches "BK001", "bk001", "Bk001"
```

0.5 Access Modifiers

Control who is allowed to read or call a field or method.

Modifier	Who can access	Example from SLCAS
public	Anyone, from any class	public boolean checkout()
private	Only the same class	private String title (LibraryItem fields)
protected	Same class + subclasses + package	(rarely used in SLCAS)
(none)	Same package only	static final int LOAN_PERIOD_DAYS

0.6 Control Flow — if, else if, else

Control flow lets your program choose different execution paths. A guard clause returns early rather than deeply nesting branches — a pattern used throughout SLCAS.

```
// From BorrowRecord.toString() — three branches:
String status;
if (returnDate != null) {
    status = "Returned " + returnDate;
} else if (isOverdue()) {
    status = "OVERDUE by " + getDaysOverdue() + " day(s)";
} else {
    status = "Due " + dueDate;
  }

// Guard clause style (LibraryItem.checkout()):
if (getAvailableCopies() <= 0) return false;  // bail out early
setAvailableCopies(getAvailableCopies() - 1); // only runs if available
return true;
```

```
// Ternary — one-line if/else: condition ? valueIfTrue : valueIfFalse
String s = (returnDate != null) ? "Returned" : "Still out";
```

Tip: Guard clauses (return early when a condition fails) keep methods short and readable. This pattern is used in checkout(), validate(), findItemById(), and many other methods in SLCAS.

0.7 Loops — for, while, for-each

Loops repeat a block of code. All three forms are used in this project.

Classic for — when you need the index

```
// for (initialise; condition; step) { body }
// From SortEngine.insertionSort():
for (int i = 1; i < n; i++) {
    LibraryItem key = list.get(i);
    int j = i - 1;
    while (j >= 0 && cmp.compare(list.get(j), key) > 0) {
        list.set(j + 1, list.get(j));  // shift right
        j--;
    }
    list.set(j + 1, key);              // insert in correct position
}
```

for-each — iterating every element in a collection

```
// for (Type variable : collection) { body }
// From LibraryDatabase.findItemById():
for (LibraryItem item : catalogue) {
    if (item != null && trimmed.equalsIgnoreCase(item.getId())) {
        recordAccess(item);
        return item;  // found — stop
    }
}
return null;
```

while — when iterations depend on a condition, not a counter

```
// while (condition) { body }
// From SearchEngine.binaryFindAny():
int low = 0, high = items.size() - 1;
while (low <= high) {
    int mid = (low + high) >>> 1;  // >>> avoids integer overflow
    String midVal = getFieldValue(items.get(mid), field).toLowerCase();
    if (midVal.contains(query))      return mid;      // found
    if (midVal.compareTo(query)<0)  low  = mid + 1;  // go right
    else                             high = mid - 1;  // go left
}
return -1;
```

0.8 Arrays vs ArrayList

Both hold multiple values of the same type, but their key difference is size: arrays are fixed, ArrayLists grow dynamically. Choosing the right one is a deliberate design decision in this project.

Feature	Array vs ArrayList

Size	Array: FIXED at creation — cannot grow or shrink.ArrayList: DYNAMIC — grows automatically when you add elements.
Syntax	Array: LibraryItem[] cache = new LibraryItem[10];ArrayList: ArrayList<LibraryItem> list = new ArrayList<>();
Read/Write	Array: cache[0] = item; int hits = accessCount[2];ArrayList: list.get(0) list.set(0, item)
Add/Remove	Array: Not supported — must create a new array.ArrayList: list.add(item) / list.remove(item) — O(1) amortised
In SLCAS	Array: accessCache[10] — frequency cache, always exactly 10 slots.ArrayList: catalogue, members list, systemLogs, changeListeners

```
// ARRAY — used for the 10-slot frequency cache in LibraryDatabase:
private LibraryItem[] accessCache = new LibraryItem[10]; // fixed 10
private int[]         accessCount = new int[10];
accessCache[0] = item;           // write by index
for (int i=0; i<accessCache.length; i++) { ... }

// ARRAYLIST — used for the catalogue:
private ArrayList<LibraryItem> catalogue = new ArrayList<>();
catalogue.add(new Book("BK001","Clean Code","R. Martin",2008));
catalogue.remove(item);
LibraryItem found = catalogue.get(3);
for (LibraryItem item : catalogue) { ... }  // for-each
```

0.9 Annotations — @Override and @SuppressWarnings

Annotations are metadata tags (prefixed with @) that instruct the compiler. They appear on the line directly above what they annotate.

@Override — guard against typos when overriding

Tells the compiler: 'this method must override a method from my parent class or interface.' If the signature doesn't match (e.g. a typo in the method name), the compiler reports an error immediately. Without @Override, a typo compiles silently as a brand-new unrelated method.

```
// In Book.java — overrides the abstract method from LibraryItem:
@Override
public String getType() { return "Book"; }

// In LibraryManager — overrides all methods from LibraryController:
@Override
public boolean borrowItem(String userId, UserAccount s, LibraryItem item){...}

// In VirtualTableModel — overrides AbstractTableModel:
@Override public int    getRowCount()    { return rows.size(); }
@Override public Object getValueAt(int r, int c) { ... }

// Without @Override, this typo would silently create a NEW method:
public String gettype() { return "Book"; } // BUG: never called by Swing!
```

@SuppressWarnings — silence known-safe compiler warnings

Suppresses a specific category of compiler warning. In SLCAS it is used exclusively with "unchecked" in FileHandler.java and LibraryManager.java when casting ObjectInputStream.readObject() (which returns Object) to a typed collection. The cast is safe because we wrote the file ourselves — the annotation signals that we have verified this manually.

```
// FileHandler.loadData():
@SuppressWarnings("unchecked")
public static List<LibraryItem> loadData() {
    try (ObjectInputStream in = new ObjectInputStream(
             new FileInputStream(DATA_FILE))) {
        return (List<LibraryItem>) in.readObject(); // cast we know is safe
    } catch (Exception e) { return new ArrayList<>(); }
}

// LibraryManager.importFromBackup():
@SuppressWarnings("unchecked")
List<LibraryItem> items    = (List<LibraryItem>) data[0];
@SuppressWarnings("unchecked")
List<UserAccount> students = (List<UserAccount>) data[1];
```

Part 1 — Object-Oriented Programming Concepts

The four OOP pillars and design patterns used in this project.

1.1 Encapsulation

Fields are private — nothing outside can change them directly. Setters validate before storing, getters provide read access. This prevents invalid state from ever entering the system.

```
private String title;  // blocked from direct outside access
public void setTitle(String title) {
    if (title == null || title.isBlank())
        throw new IllegalArgumentException("Title cannot be empty.");
    this.title = title.trim();  // this.title = field; title = parameter
  }
public String getTitle() { return title; }
// LibraryItem has 5 validated setters: setId, setTitle, setAuthor,
// setYear (range 1000-nextYear), setTotalCopies (preserves borrowed count)
```

1.2 Inheritance

Child classes reuse all non-private fields and methods of their parent automatically. super() passes values up to the parent's constructor.

```
public abstract class LibraryItem {    // parent
    private String id, title, author;  // shared by all item types
    public abstract String getType();  // children MUST implement this
  }
public class Book     extends LibraryItem { @Override public String
getType(){return"Book";}    }
public class Magazine extends LibraryItem { @Override public String
getType(){return"Magazine";} }
public class Journal  extends LibraryItem { @Override public String
getType(){return"Journal";}  }
// All three inherit: id, title, author, year, totalCopies, availableCopies,
// checkout(), checkin(), isAvailable(), all getters and validated setters
```

1.3 Polymorphism

The same method call behaves differently depending on the actual runtime type. One catalogue ArrayList can store all three item types because all ARE LibraryItems.

```
LibraryItem item = new Book("BK001", "Clean Code", "R. Martin", 2008);
System.out.println(item.getType());   // Book
item = new Magazine("MG001", "IEEE Spectrum", "IEEE", 2024);
System.out.println(item.getType());   // Magazine

// Catalogue exploits this — one list, three types:
ArrayList<LibraryItem> catalogue = new ArrayList<>();
catalogue.add(new Book(...));   catalogue.add(new Magazine(...));
catalogue.add(new Journal(...)); // all fit — all ARE LibraryItems
```

1.4 Abstract Classes vs Interfaces — Side by Side

Feature	Abstract Class vs Interface

Instance fields	Abstract class: Yes — private/protected fields allowed.Interface: No — only static final constants.
Constructors	Abstract class: Yes.Interface: No.
Method bodies	Abstract class: Yes — concrete + abstract methods.Interface: Traditionally no — only method signatures.
A class can...	Abstract class: extend ONE only.Interface: implement MANY simultaneously.
Keyword	abstract class Foo extends Bar { }class Foo implements Bar, Baz { }
In SLCAS	Abstract: LibraryItem (6 fields + concrete checkout/checkin).Interface: Borrowable, LibraryController, LibraryChangeListener, AuthController

1.5 Design Patterns

Pattern	Where in SLCAS	What it does
Singleton	AuthManager	One shared instance via getInstance() — prevents two staff stores
Observer	LibraryChangeListener	Panels auto-notified via fireChange() after every data mutation
Strategy	SortEngine / SearchEngine	Different algorithms swapped at runtime without changing GUI code
Dep. Inversion	LibraryController interface	GUI holds the interface, never the concrete LibraryManager
Template Method	LibraryItem abstract class	Parent defines checkout/checkin; subclasses only implement getType()
DTO	LoanView, BorrowSummary...	Read-only data containers passed controller → GUI — no logic inside

Part 2 — Project Structure & Architecture

2.1 Package Overview

Package	Contents
model/	LibraryItem (abstract), Book, Magazine, Journal, Borrowable (interface), UserAccount, BorrowRecord, LibraryDatabase, WaitlistEntry (record), SystemLog, BorrowSummary, StudentSummary, LoanView, OverdueLoanView
controller/	LibraryController (interface), LibraryManager, BorrowController, SearchEngine, SortEngine, LibraryChangeListener (interface), AuthController (interface)
gui/	MainWindow, LoginDialog, ViewPanel, AdminPanel, BorrowPanel, StudentPanel, SearchSortPanel, StaffManagementPanel, LogsPanel, VirtualTableModel, GuiUtils
utils/	FileHandler, AuthManager (Singleton + SHA-256), IDGenerator (UUID-based), DataSeeder (25 demo items + 10 students)
main/	LibraryApp.java — the only class that contains main()

2.2 Application Startup — main() to MainWindow

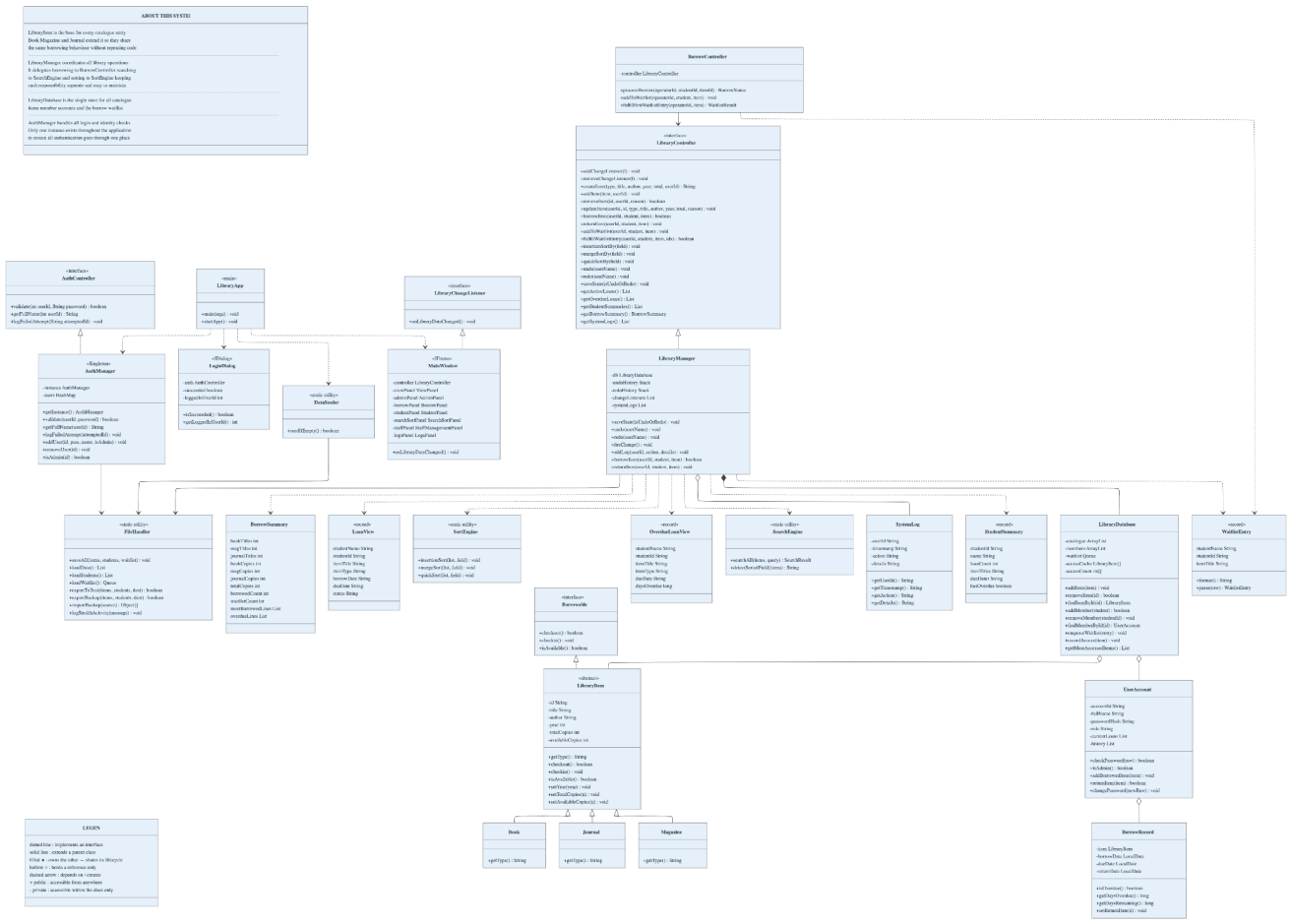
```
// main/LibraryApp.java
public static void main(String[] args) {
    UIManager.setLookAndFeel(getCrossPlatformLookAndFeelClassName()); // 1
    AuthManager.loadUsers();        // 2. load staff from system_users.dat
    DataSeeder.seedIfEmpty();       // 3. seed 25 items + 10 students if empty
    SwingUtilities.invokeLater(() -> {  // 4. schedule on Swing EDT
        LoginDialog login = new LoginDialog(null, AuthManager.getInstance());
        login.setVisible(true);     // 5. show login — blocks until closed
        if (login.isSucceeded())
            new MainWindow(login.getLoggedInUserId()); // 6. open main window
        else
            System.exit(0);         // 7. exit if login cancelled
    });
}
```

2.3 Login Accounts (seeded by AuthManager + DataSeeder)

Role	Staff ID	Password & Notes
Admin (Super)	30114413	Ol@l3r3 — full access, cannot be deleted (SUPER_ADMIN_ID constant)
Admin	1002	Admin456 — full access including staff management and system logs
Librarian	1001	Staff123 — restricted: borrow/return, catalogue view, search only

2.4 UML Class Hierarchy & Relationships

The diagram uses standard ASCII characters: + (corners), - (horizontal lines), | (vertical lines), ^ (arrow pointing up for extends/implements), --> (uses/depends on). Read top-to-bottom — interfaces at Layer 1, concrete classes at Layers 2–3, controllers and utilities at Layer 4, GUI panels at Layer 5.



Part 3 — Code Walkthrough (File by File)

3.1 model/LibraryItem.java — Abstract Base

- abstract class — new LibraryItem() is a compile-time error. Must use Book, Magazine, or Journal.
- implements Serializable — enables binary save/load to .dat files via ObjectOutputStream.
- implements Borrowable — provides concrete bodies for checkout(), checkin(), isAvailable().
- All setters validate input: blank strings throw IllegalArgumentException; year must be 1000–(thisYear+1); copies cannot be negative.
- setTotalCopies(n): calculates borrowed = totalCopies − availableCopies, then sets availableCopies = max(0, n − borrowed). Preserves the borrowed count when an admin edits the quantity.
- checkout(): returns false immediately if availableCopies <= 0. Otherwise decrements by 1 and returns true.

3.2 model/LibraryDatabase.java — Central Data Store

Field	Type	Purpose
catalogue	ArrayList<LibraryItem>	All items. Index access required by all 3 sort algorithms.

members	ArrayList<UserAccount>	All students. Stream-searched by ID in findMemberById().
waitlist	Queue<String> (LinkedList)	FIFO. offer() enqueues. BorrowController polls/searches it.
accessCache	LibraryItem[10] + int[10]	Fixed-size frequency cache. Sorted descending by hit count.

recordAccess(): increments existing entry count, adds to first empty slot, or evicts index 9 (lowest after sort) when full. sortCacheDescending() uses insertion sort on both arrays in tandem. getMostAccessedItems() returns ranked non-null entries for the ViewPanel Popular section.

3.3 model/UserAccount.java — All User Types

- One class covers Admin, Librarian, and Student. The role string field controls privileges; isAdmin() returns true only when role.equals("Admin").
- Passwords: hashPassword(raw) applies SHA-256 + Base64 encoding. checkPassword(raw) hashes input and compares. The original password is never recoverable from the stored hash.
- currentLoans (List<BorrowRecord>): active loans. history (List<BorrowRecord>): returned loans.
- returnItem(item): iterates currentLoans, stamps returnDate = LocalDate.now(), removes from currentLoans, appends to history. Returns false if item not found in current loans.
- Copy constructor UserAccount(UserAccount other): deep-copies all BorrowRecords via BorrowRecord's own copy constructor. Used by the undo/redo snapshot system to ensure snapshots are fully independent from live data.

3.4 model/BorrowRecord.java — One Loan Transaction

- Stores: item reference, borrowDate, dueDate (borrowDate + LOAN_PERIOD_DAYS = 14), returnDate (null while still borrowed).
- isOverdue(): returnDate == null AND LocalDate.now().isAfter(dueDate).
- getDaysOverdue(): uses ChronoUnit.DAYS.between(dueDate, now). Returns 0 if not overdue.
- Copy constructor BorrowRecord(BorrowRecord other): used by UserAccount's copy constructor for undo/redo deep copies.

3.5 model/WaitlistEntry, DTOs & SystemLog

- WaitlistEntry is a record. format() produces 'Name (ID) -> Title'. parse(raw) is a static factory that returns null on bad format.
- LoanView, OverdueLoanView, StudentSummary are records — read-only data containers passed from controller to GUI.
- BorrowSummary: all-final public fields covering title/copy counts, most-borrowed and overdue text lines for the Reports dialog.
- SystemLog: immutable audit entry. Timestamp is auto-generated at construction time with LocalDateTime.now().

3.6 controller/LibraryController.java — The Interface

Declares ~40 methods covering item CRUD, student CRUD, borrow/return, waitlist, all three sort algorithms, undo/redo, audit logging, staff management, observer management, and DTO queries. All GUI panels hold a LibraryController reference — never the concrete LibraryManager. This is the Dependency Inversion Principle: if LibraryManager is ever replaced with a different implementation, no GUI code needs to change.

3.7 controller/LibraryManager.java — Main Controller

- Implements LibraryController and java.io.Serializable. Constructor loads all .dat files and calls saveState(false) to seed the undo stack.
- Observer: changeListeners is transient (not serialised). fireChange() iterates a defensive copy (new ArrayList<>(changeListeners)) to avoid ConcurrentModificationException.
- Undo/Redo: LibraryState inner class stores deep-copied catalogue, members, and waitlist. saveState() pushes to undoHistory before every mutation. undo() pops undoHistory, saves current to redoHistory, restores. redo() reverses.
- borrowItem(): prevents duplicate loans with stream().anyMatch() on currentLoans. Calls item.checkout(), student.addBorrowedItem(), saves all .dat files, fires change.
- returnItem(): calls item.checkin(), student.returnItem(), saves, fires change, then calls BorrowController.fulfilFirstWaitlistEntry() to auto-serve the next waiting student.

3.8 controller/SearchEngine.java — Auto-Selecting Search

```
// searchAll() auto-picks the algorithm:
String sortedField = detectSortedField(items); // null if unsorted
if (sortedField != null)
    return new SearchResult(binarySearchAll(items, query, sortedField),
                            "Binary Search (sorted by " + sortedField + ")");
return new SearchResult(linearSearchAll(items, query),
                        "Linear Search (unsorted)");
// linearSearchAll — O(n): checks ID, Title, Author, Type, Year per item
// binarySearchAll — O(log n) on sorted field, then expands left+right
```

3.9 controller/SortEngine.java — Three Algorithms

- All methods are static — pure utility, no state. comparatorFor(field) uses modern switch expression; string fields compared case-insensitively.
- insertionSort: $O(n^2)$ worst, $O(n)$ best (nearly-sorted). Shifts each 'key' element left until it reaches its correct sorted position.
- mergeSort: $O(n \log n)$ all cases. Splits at mid = left+(right−left)/2 (avoids integer overflow). merge() copies both halves to L[] and R[] arrays, then merges back.
- quickSort: $O(n \log n)$ average. Median-of-three pivot — compares first, middle, last elements and swaps so median is at the high index. Partitions in-place. Worst-case $O(n^2)$ is extremely unlikely with this strategy.

3.10 utils/FileHandler.java — All Disk I/O

- Saves catalogue, students, and waitlist to .dat binary files via ObjectOutputStream in try-with-resources (streams auto-closed even on exception).
- Load methods return empty collections on missing or corrupted files — the app never crashes on bad data. All use @SuppressWarnings("unchecked") for the ObjectInputStream cast.
- exportToText(): human-readable stock summary + full inventory + student records via PrintWriter.
- logStealthActivity(): appends timestamped lines to sys_debug_core.log. Empty catch block — a log failure must never crash the app.

3.11 utils/AuthManager.java — Singleton Pattern

```
public class AuthManager implements AuthController {
    public static final int SUPER_ADMIN_ID = 30114413;
    // Singleton — one instance, created once at class-load time
    private static final AuthManager INSTANCE = new AuthManager();
    private AuthManager() {}  // private — blocks 'new AuthManager()'
```

```
    public static AuthManager getInstance() { return INSTANCE; }
    // Map<Integer staffId, UserAccount>
    private static Map<Integer, UserAccount> users = new HashMap<>();
    // SHA-256 + Base64 — passwords never stored as plain text
    public static String hashPassword(String raw) {
        byte[] hash=MessageDigest.getInstance("SHA-256").digest(raw.getBytes("UTF-8"));
        return Base64.getEncoder().encodeToString(hash);
    }
}
```

3.12 gui/VirtualTableModel.java — Lazy Rendering

- Extends AbstractTableModel. Backed by List<Object[]> rows built by GuiUtils.buildCatalogueRows(). Column names defined in GuiUtils.CATALOGUE_COLUMNS = {"ID","Type","Title","Author","Year","Available","Total"}.
- JTable calls getValueAt(row, col) ONLY for visible rows — off-screen data is never processed. Keeps the UI fast with hundreds of items.
- setRows(newRows): replaces all rows atomically and calls fireTableDataChanged() for a single repaint.
- isCellEditable() always returns false — editing happens through form fields, not inline in the table.

Part 4 — Data Structures: Why Each One Was Chosen

This is a high-probability exam topic. Know not just WHAT data structure is used, but WHY it was the correct choice for that specific use case.

4.1 ArrayList — Catalogue & Member List

- Chosen because: items need to be added, removed, iterated, and sorted in-place by index.
- O(1) index access via get(i) and set(i) is essential for all three sort algorithms.
- O(1) amortised add to end; O(n) remove by value — acceptable for library sizes.

4.2 Queue (LinkedList) — The Waitlist

- Chosen because: a reservation queue is FIFO — the first student to join must be first served.
- offer() adds to the tail O(1). LinkedList is efficient; ArrayList.remove(0) would be O(n).
- After a return, BorrowController.fulfilFirstWaitlistEntry() finds the first matching entry, borrows for that student, and removes the entry.

4.3 Stack — Undo / Redo

- Chosen because: undo/redo is LIFO — the most recent action is first to be undone.
- undoHistory.push() snapshots before every mutation. undo() pops the most recent, pushes current to redoHistory, restores. redo() reverses.

```
undoHistory.push(new LibraryState(catalogue, members, waitlist));
LibraryState prev = undoHistory.pop(); redoHistory.push(current);
```

4.4 Fixed Array — The Frequency Cache

- Chosen because: size is ALWAYS exactly 10 — a fixed array is correct and more efficient than ArrayList.
- Parallel int[] tracks hit counts. Both arrays sorted together descending by insertion sort after every access. Index 9 (lowest count) is evicted when cache is full.

4.5 HashMap — Staff Accounts

- Chosen because: login needs O(1) average lookup by staff ID (integer key). No ordering needed — HashMap is optimal.
- Map<Integer, UserAccount>: staffId → account. Persisted to system_users.dat via ObjectOutputStream.

Part 5 — Likely Lecturer Questions & Model Answers

These cover the most probable questions for a viva or written assessment. Study both the question and the full reasoning — examiners usually want the 'why', not just the 'what'.

Question 1 of 15

Q: What is the entry point? Walk through what happens when the app starts.

A: main/LibraryApp.java contains public static void main(String[] args). Step 1: Swing look-and-feel is set. Step 2: AuthManager.loadUsers() reads system_users.dat — creates a default Super Admin (ID 30114413, password Ol@l3r3) if the file doesn't exist. Step 3: DataSeeder.seedIfEmpty() generates 25 demo items (15 books BK001–BK015, 5 magazines MG001–MG005, 5 journals JN001–JN005) and 10 students if the .dat files are empty or unreadable. Step 4: SwingUtilities.invokeLater() schedules execution on the Swing Event Dispatch Thread. Step 5: LoginDialog is shown. Step 6: on success, new MainWindow(userId) opens the main application; otherwise System.exit(0).

Question 2 of 15

Q: Why is LibraryItem abstract? What compile-time error occurs if you try to instantiate it?

A: LibraryItem is abstract because it represents a concept — you cannot hold a generic 'library item', only a Book, Magazine, or Journal. The abstract keyword makes the compiler refuse new LibraryItem() with: 'LibraryItem is abstract; cannot be instantiated'. It also declares getType() as abstract, meaning every subclass MUST provide its own implementation. Without this, the 'Type' column in the catalogue table would have no value and all three subclasses would be indistinguishable in the GUI.

Question 3 of 15

Q: Explain the difference between extends and implements. Give examples from the project.

A: extends is for class inheritance — a child class gains all non-private fields and methods of the parent. A class can extend only ONE other class.
implements is for interfaces — a class promises to provide method bodies for every signature declared in the interface. A class can implement many interfaces simultaneously.
In SLCAS: Book extends LibraryItem. LibraryItem implements Borrowable AND Serializable. LibraryManager implements LibraryController AND java.io.Serializable. GUI panels implement LibraryChangeListener. This means a panel can extend JPanel (one class) and implement LibraryChangeListener (one interface) at the same time.

Question 4 of 15

Q: What is the MVC pattern and how does SLCAS implement it?

A: MVC (Model-View-Controller) separates data, logic, and display into three independent layers. Model (model/ package): pure data objects — LibraryItem, UserAccount, BorrowRecord, LibraryDatabase. Controller (controller/): processes operations and enforces business rules — LibraryManager, BorrowController, SearchEngine, SortEngine. View (gui/): Swing panels that display data and accept user input — MainWindow, AdminPanel, BorrowPanel etc.
The critical rule: all GUI panels store a LibraryController interface reference, never the concrete LibraryManager. This enforces the Dependency Inversion Principle: the View depends on an abstraction, not a concrete implementation.

Question 5 of 15

Q: What is the Observer pattern and exactly how does it work in SLCAS?

A: Observer allows objects (observers) to be automatically notified when another object (the subject) changes state. LibraryChangeListener is the observer interface with one method: onLibraryDataChanged(). Every GUI panel implements it and calls controller.addChangeListener(this) when constructed. LibraryManager maintains a List<LibraryChangeListener> changeListeners.

After every mutation — add item, remove item, borrow, return, sort, undo, redo — LibraryManager.fireChange() iterates a defensive copy (new ArrayList<>(changeListeners)) calling onLibraryDataChanged() on each panel. Each panel then calls GuiUtils.buildCatalogueRows() and updates its VirtualTableModel. The defensive copy prevents ConcurrentModificationException if a listener removes itself during notification.

Question 6 of 15

Q: Why are there three sorting algorithms? Compare their time complexities and stability.

A: Having all three lets users observe different algorithm behaviours directly, and each has distinct trade-offs. Insertion Sort: O(n) best case on nearly-sorted data, O(n^2) average and worst, stable — best for small or partially-sorted catalogues. Merge Sort: O(n log n) guaranteed in ALL cases, stable, but uses O(n) extra memory for temporary L[] and R[] arrays — best when correctness of order matters most. Quick Sort: O(n log n) average, in-place (no extra memory), NOT stable, O(n^2) worst case — mitigated here by median-of-three pivot selection. All three sort the same ArrayList<LibraryItem> in-place using Comparators returned by SortEngine.comparatorFor(field).

Question 7 of 15

Q: How does undo/redo work? What data structure is used and why?

A: Stack is used because undo/redo is LIFO — the most recently performed action is the first to be undone. LibraryManager has undoHistory and redoHistory Stacks of LibraryState (a private inner class holding shallow copies of catalogue/waitlist and deep copies of the members list).

Before every mutation, saveState() creates a new LibraryState — deep-copying each UserAccount using its copy constructor, which deep-copies each BorrowRecord — and pushes to undoHistory. undo(): pops the top of undoHistory, pushes current state onto redoHistory, restores the database from the popped state, fires change. redo(): reverses. The deep copy is critical: without it, changes to live BorrowRecord objects would corrupt the snapshot retroactively.

Question 8 of 15

Q: What is serialization? What must a class do to support it? What happens if the file is corrupted?

A: Serialization converts a Java object graph into a byte stream for disk storage. In SLCAS all catalogue, student, and waitlist data is serialized to .dat binary files so it persists between restarts. A class must: 1) implement Serializable, 2) declare private static final long serialVersionUID (a version tag the JVM checks at deserialization).

If a .dat file is corrupted — wrong type, incompatible serialVersionUID, truncated bytes — ObjectInputStream.readObject() throws an exception. FileHandler's catch block returns an empty collection, and DataSeeder.seedIfEmpty() then re-generates demo data. The failure is logged to sys_debug_core.log by logStealthActivity(). The app degrades gracefully rather than crashing.

Question 9 of 15

Q: How are passwords stored and verified? Why not store them as plain text?

A: UserAccount.hashPassword(raw) applies SHA-256 — a cryptographic one-way function — to the raw password bytes and encodes the 32-byte result as a Base64 string (~44 characters). Only this hash is stored in system_users.dat. At login: AuthManager.validate() calls UserAccount.checkPassword(raw) which re-hashes the typed password and compares to the stored hash with String.equals().

Storing plain text is dangerous: if system_users.dat were stolen, all passwords would be immediately exposed. SHA-256 is computationally infeasible to reverse — even with the hash, an attacker cannot recover the original password.

Question 10 of 15

Q: What is the Singleton pattern? How is it implemented in AuthManager and why is it needed?

A: Singleton ensures exactly one instance of a class exists throughout the application's lifetime. AuthManager uses three rules: 1) private constructor — new AuthManager() is blocked from any external code. 2) private static final INSTANCE = new AuthManager() — created exactly once at class-load time, inherently thread-safe due to JVM class-loading guarantees. 3) public static getInstance() — always returns the same INSTANCE.

Without Singleton, two different AuthManager objects could be created with different user stores. Staff credentials could be loaded into one and checked against the other, making login always fail for valid users — a hard-to-debug race condition.

Question 11 of 15

Q: Explain the waitlist end to end — data structure, format, add, and fulfilment after a return.

A: Data structure: Queue<String> backed by LinkedList — FIFO ensures the first student to join is first served. Format: each entry is a String formatted as 'Name (ID) -> ItemTitle' by WaitlistEntry.format(). Adding: addToWaitlist() builds the entry and calls db.enqueueWaitlist(entry.format()) which calls Queue.offer() — O(1).

Fulfilment: after every successful return, LibraryManager.returnItem() calls BorrowController.fulfilFirstWaitlistEntry(item). This streams the queue filtering for entries where WaitlistEntry.parse(entry).itemTitle() matches the returned item's title. If found: borrowItem() auto-borrows for that student, then removeWaitlistEntry() converts the queue to a List, removes by index, and rebuilds. The student is notified via an audit log entry.

Question 12 of 15

Q: What are DTOs? List all DTOs in SLCAS and explain why they are used.

A: DTO (Data Transfer Object) is a read-only container that carries data between layers without exposing internal model classes to the GUI. They decouple the View from the Model.

In SLCAS: LoanView (one active loan row — studentName, itemTitle, borrowDate, dueDate, status), OverdueLoanView (one overdue entry — student, item, dueDate, daysOverdue), StudentSummary (one student table row — includes pre-computed hasOverdue flag so the renderer does no date arithmetic), BorrowSummary (all statistics for the Reports dialog — title/copy counts, mostBorrowedLines, overdueLines), SystemLog (immutable audit entry — userId, timestamp, action, details). LoanView, OverdueLoanView, and StudentSummary are Java records; BorrowSummary and SystemLog use all-final fields.

Question 13 of 15

Q: What would happen if you omitted @Override on Book.getType()? What about on a GUI panel?

A: On Book.getType(): if you mistype the name — e.g. gettype() — without @Override the compiler silently creates a brand-new method, leaving LibraryItem's abstract getType() unimplemented. The class would fail to compile anyway because it extends an abstract class without implementing all abstract methods. But in a non-abstract class with a concrete parent, a typo would compile silently and the correct override would never execute.

On a GUI panel implementing LibraryChangeListener: if onLibraryDataChanged() were misspelled (e.g. onLibraryDatachanged()), without @Override it would compile as a new unrelated method. The panel would register via addChangeListener() but would never be called by fireChange(), so the panel would never refresh after data changes — a silent, hard-to-find bug.

Question 14 of 15

Q: Explain the access frequency cache. What problem does it solve and how does it work?

A: Problem: the GUI needs to show users which items are most popular without scanning the entire catalogue every time. Solution: a fixed-size cache of the 10 most-frequently accessed items.

Implementation in LibraryDatabase: two parallel arrays — LibraryItem[] accessCache and int[] accessCount, both size 10. Every time findItemById() finds an item, recordAccess(item) is called. It checks if the item is already in the cache (increment its count), then adds it to the first empty slot (if cache has space), or replaces index 9 (lowest count after descending sort) if the cache is full. sortCacheDescending() uses insertion sort to keep both arrays aligned and sorted — O(n) which is fine for n=10. getMostAccessedItems() returns the non-null entries in ranked order for the ViewPanel.

Question 15 of 15

Q: What is the Dependency Inversion Principle? How does SLCAS demonstrate it?

A: DIP states: high-level modules should depend on abstractions (interfaces), not on concrete implementations. This prevents tight coupling where a change in one class forces changes in all classes that depend on it.

In SLCAS: every GUI panel stores a LibraryController reference (the abstraction), never a LibraryManager reference (the concrete class). MainWindow constructs one LibraryManager instance and immediately casts it: this.controller = new LibraryManager(). After that, all panels receive a LibraryController. Similarly, BorrowController holds a LibraryController, not LibraryManager. If LibraryManager were ever replaced with a NetworkLibraryManager for a server-based version, no GUI code would need to change — only the one line in MainWindow where the concrete type is constructed.